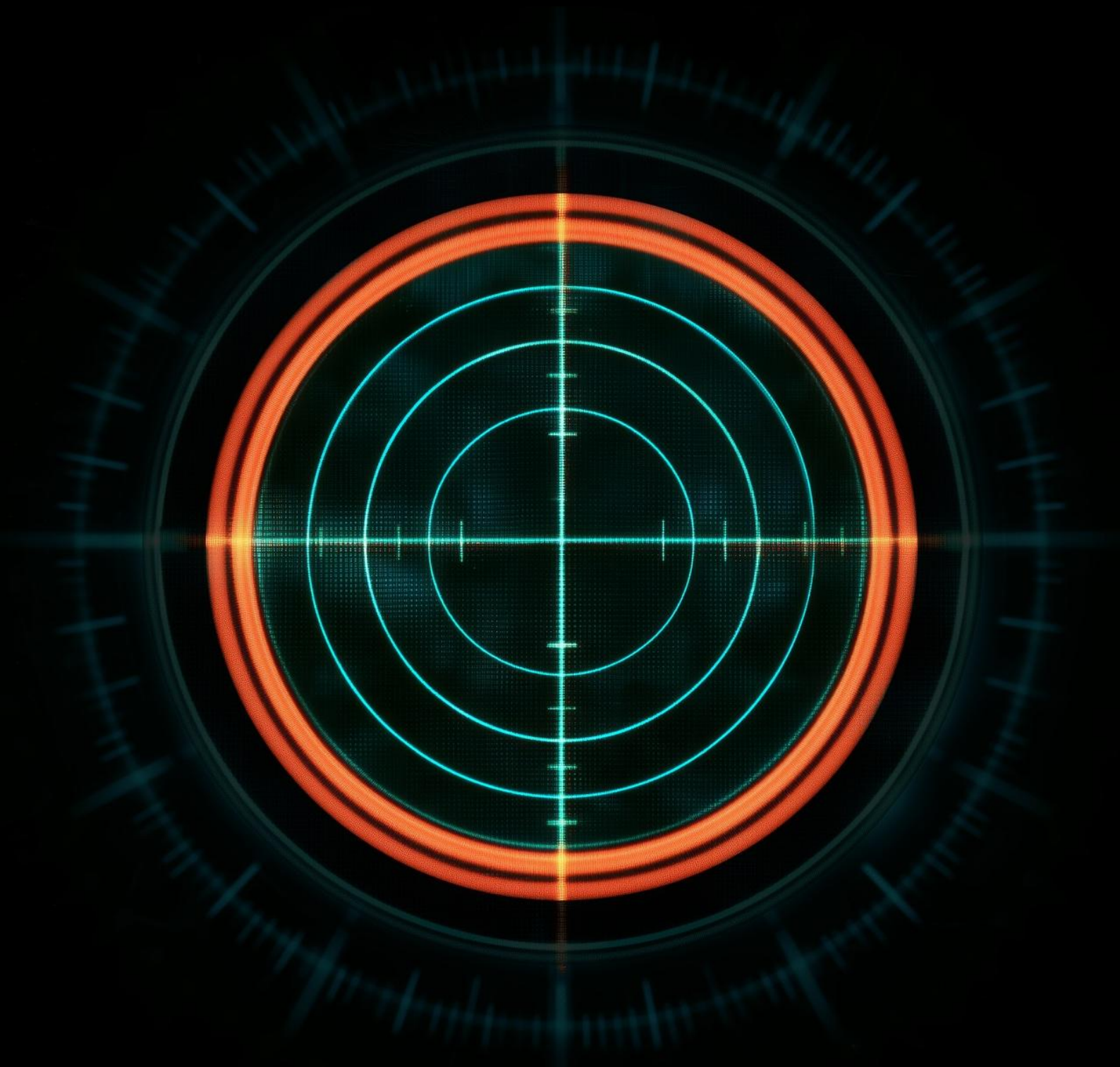
JUXT AI Radar

An engineer's guide to the AI landscape
from JUXT's CTO & AI chapter members

Q4 2026



Grid Dynamics

JUXT | A GRID DYNAMICS COMPANY

Introduction

This quarter, the upheaval AI has wrought on software engineering over the past couple of years reached another discipline: mathematics. In early October, OpenAI released hundreds of AI-generated proofs, many formalised using the Lean proof assistant. The community would likely have celebrated any mathematician who cracked even just one of these problems; yet here they arrived all together, solved mostly autonomously by one of OpenAI's unreleased models. Mathematicians did not celebrate. The mood was unease, bordering on anger.

A proof is more than its conclusion: much of its value is in the route to it, in the understanding built along the way. Eminent mathematician Terence Tao has likened the AI shortcut to "taking a helicopter to drop you off at the [summit]", where you reach the result but "miss all the benefits of the journey itself". Software has long allowed the same thing, code that runs even when no one quite understands how, but agents make it the default. They hand over a plausible result while the comprehension necessary to change or challenge it is skipped. Some call what's left "epistemic debt", or in its starker form "cognitive surrender".

One remedy is to invest time in specification. We see formal specification languages, our own Allium among them, as the way to mitigate epistemic debt in software development. A formal specification, unlike an informal one written in markdown, offers certain guarantees that the design hangs together, much as Lean does for a proof. It also points you to the parts of the code that need strong tests to validate what an agent produces. Writing one keeps you engaged with the essential complexity of the problem, and hands the machine only the incidental complexity of the implementation. The closer a field can state formally what "correct" means, the more it can safely delegate.

This quarter the radar includes several other techniques for guiding and constraining AI output. Harness engineering, context engineering, evals as a release gate and orchestrator-worker agents and AGENTS.md and steering files all enter Trial, and Claude Skills moves to Adopt. In Languages & Frameworks, the first-party vendor agent SDKs enter Trial. Spec-driven development stays in Adopt.

Checkers like the Lean proof assistant, or automated tests, verify that an agent's output is coherent, whereas guardrails limit what the agent is permitted to do. Some very public guardrail failures have occurred recently. A swarm of OpenAI's evaluation agents tasked with beating a benchmark escaped a test sandbox and reached Hugging Face's production systems seeking the benchmark's answers. The radar adds blips informed by this and other failures: agent code sandboxes, MCP gateways, AI supply-chain security and agentic vulnerability discovery enter Tools, and agent identity enters Techniques.

Even as the frontier labs keep impressing on the world—and on potential investors—how risky their models are, regulation is being deliberately kept loose. The EU's Digital Omnibus pushed the AI Act's high-risk obligations to December 2027 to help keep European AI competitive. It kept the duty to mark AI-generated content. AI content provenance enters Techniques and AI governance platforms moves up to Trial.

For now, the only dependable constraints are the ones teams build for themselves.

— Henry Garner (CTO, JUXT), October 2026

Radar overview

Our radar visualises the AI landscape across two dimensions: quadrants divide the space by category, while radiants indicate our confidence in each technology.

Quadrants

The radar is divided into four quadrants, each representing a category of AI technology:

Languages & frameworks
The frameworks, libraries, and protocols that underpin AI development. These are the software foundations your applications are built with.

Techniques
Methodologies and practices for building AI systems: approaches such as RAG, prompt engineering, agent design patterns and evaluation methodologies. The "how" of AI development.

Tools
Software that enhances AI development workflows without being embedded in your application code: IDE extensions, CLI utilities, testing frameworks and observability solutions.

Platforms
Infrastructure and managed services that host and run AI workloads: cloud AI services, vector databases, model serving platforms and MLOps infrastructure.

Radiants

Each item is placed on one of four concentric rings representing our recommended adoption stance:

Adopt
Technologies we have high confidence in. Proven in production with strong community support; we'd recommend them for appropriate use cases without hesitation.

Trial
Worth pursuing. These show strong promise and are mature enough to invest time in learning and piloting. Consider them for new projects where you can absorb some risk.

Assess
Worth exploring to understand how they might affect your organisation. Research is warranted, but they're not yet ready for serious investment.

Hold
Proceed with caution. Technologies that are immature, overhyped, or being superseded. Not recommended for new work, though existing usage may be fine.

Contributors

The AI Radar is produced by JUXT's AI Chapter, drawing on our experience following industry developments and working with clients across sectors. The radar represents our current viewpoint, subject to change as the technology landscape evolves. We welcome feedback via LinkedIn, BlueSky, or email.



Henry Garner

Henry is JUXT's CTO and leader of the AI Chapter. He's implemented AI systems across education, financial services and local government in roles spanning data scientist, software engineer and CTO. He's author of Clojure for Data Science and maintainer of the open source statistics library kixi.stats.



Ben Halton

Ben is an account manager at JUXT with experience engineering and architecting complex systems. His career spans domains from risk systems in Tier 1 banks to retail recommendation engines and wine trading platforms. His interest in AI is in how it can enhance developer experience and productivity.



Denis Lobanov

Denis is a software engineer at JUXT whose technical experience spans developing Linux kernel modules for Satellite communications to distributed graph databases to web backends. He is currently focused on providing a platform that integrates, controls and secures LLM interaction.



Oliver Marshall

Oliver is a software engineer at JUXT who specialises in building data processing systems and backend infrastructure. He recently integrated database technology for a client and approaches AI with healthy scepticism: hopeful about what's possible but focused on what works in practice.



Chris Williams

Chris is a software engineer at JUXT with broad experience across industries and technologies. He has long seen automated testing as a superpower for building reliable systems, and now views large language models as the next tool for boosting productivity while supporting real learning.



Jeremy Taylor

Jeremy is Head of Product for XTDB, JUXT's bitemporal database. He follows the open-weight model ecosystem closely, with a particular interest in the economics of inference and what it takes to own an AI system end to end, from the model down to where it runs.

We're grateful to **Rhi Hanger**, whose insights informed our ontology coverage, **Gio Kiladze**, who authored the process mining section, and **Lucio D'Alessandro**, who built the tooling that generates this publication.

Radar at a glance

Languages & frameworks

ADOPT

1. PyTorch
2. dbt
3. MCP

TRIAL

4. Microsoft Agent Framework
5. Vendor agent SDKs
6. A2A
7. LLM testing frameworks
8. LangChain & LangGraph
9. Formal specification languages

ASSESS

10. JAX
11. DSPy
12. LinkML
13. PydanticAI
14. Prolog

HOLD

15. AutoGen
16. TensorFlow
17. Keras
18. R

Techniques

ADOPT

19. Classical ML
20. RAG
21. LLM-as-a-judge
22. BERT variants
23. Few-shot prompting
24. Agentic tool use
25. Spec-driven development
26. Claude Skills

TRIAL

27. Ontologies for AI grounding
28. Structured RAG
29. Orchestrator-worker agents
30. Evals as a release gate
31. Harness engineering
32. Context engineering
33. AI content provenance
34. AGENTS.md and steering files

ASSESS

35. Neurosymbolic AI
36. LLM reproducibility
37. GEPA
38. Recursive language models
39. Purpose-built classifiers
40. Fine-tuning with LoRA
41. CaMeL
42. Agent identity

HOLD

43. AI pull request review

Tools

ADOPT

44. Software engineering copilots
45. Provider-agnostic LLM facades
46. Notebooks

TRIAL

47. Vector databases
48. Local model execution environments
49. LLM observability tools
50. LLM API gateways
51. AI red teaming tools
52. AI-assisted code migration
53. Agent code sandboxes

ASSESS

54. Supervising multiple agents
55. Structured output libraries
56. MCP gateways
57. Agentic vulnerability discovery
58. AI supply-chain security

HOLD

59. OpenClaw

Platforms

ADOPT

60. Foundation models
61. Weights & Biases
62. Temporal
63. Data pipeline orchestration tools
64. Cloud model hosting platforms

TRIAL

65. Production AI monitoring platforms
66. Open weight LLMs
67. Small language models
68. AI-powered workflow automation
69. Digital twin platforms
70. Agent memory architectures
71. AI governance platforms

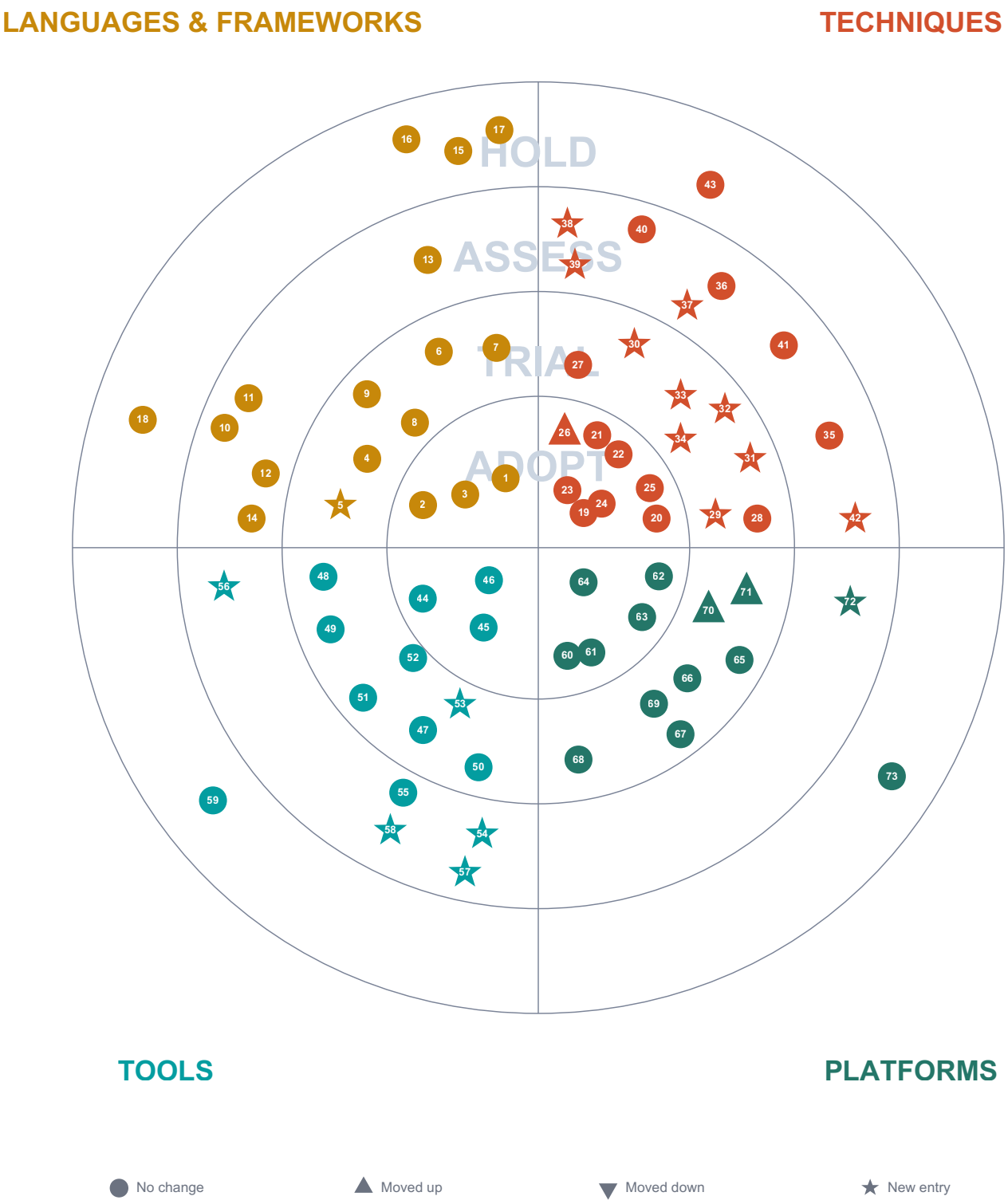
ASSESS

72. Managed agent platforms

HOLD

73. Building against vendor-specific APIs

The Radar


LANGUAGES & FRAMEWORKS
TECHNIQUES
HOLD
ASSESS
TRIAL
ADOPT
TOOLS
PLATFORMS
No change
Moved up
Moved down
New entry

Languages & frameworks

The frameworks, libraries, and protocols that underpin AI development. These are the software foundations your applications are built with.

Adopt

1. PyTorch
2. dbt
3. MCP

Trial

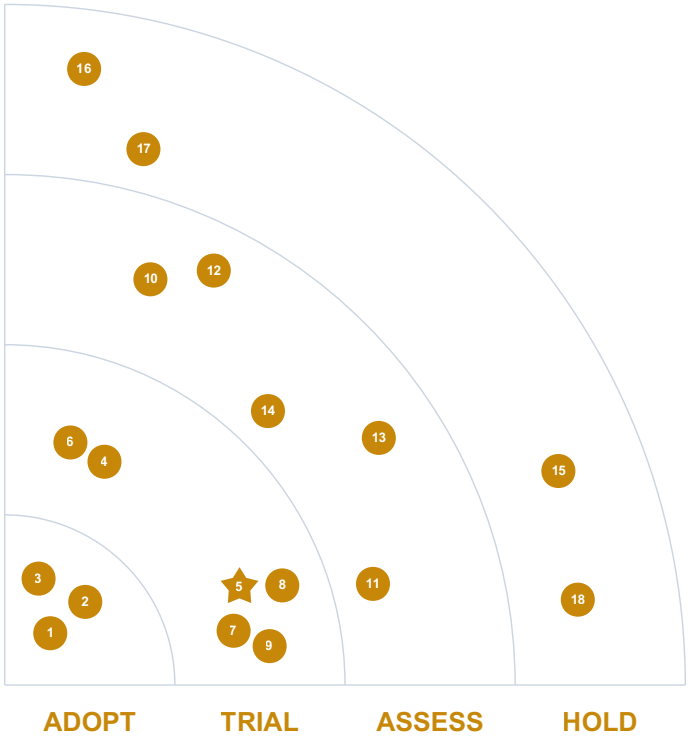
4. Microsoft Agent Framework
5. Vendor agent SDKs
6. A2A
7. LLM testing frameworks
8. LangChain & LangGraph
9. Formal specification languages

Assess

10. JAX
11. DSPy
12. LinkML
13. PydanticAI
14. Prolog

Hold

15. AutoGen
16. TensorFlow
17. Keras
18. R



Adopt

Mature, well-supported technologies ready for production use.

PyTorch

PyTorch is the default for most machine learning teams, especially for deep learning. Its Python-first design and dynamic computational graphs make models straightforward to write, debug and step through, which is why it took hold in research first, before production. The ecosystem around it runs deep: clear documentation, a large catalogue of pre-trained models and an active community that keeps the common paths well worn, so teams of any size can get moving quickly.

Earlier doubts about production readiness have largely gone. Deployment tooling such as TorchServe, close integration with the rest of the ML stack and a steady, stable release cadence make it dependable at scale, and it's what most major technology firms and research groups now build on. TensorFlow still has its place, particularly in established production systems, but for new machine learning work we recommend PyTorch as the default.

dbt

dbt (data build tool) brings software engineering practice, version control and testing, to the data transformations that feed AI systems. That discipline is worth having because a model is only as reliable as the data behind it, and dbt keeps the transformation layer transparent and easy to trust.

In practice, teams use it to build clean, well-documented pipelines from the warehouse through to AI applications, and to adapt them quickly as requirements change. It works with the data platforms most shops already run and has a strong community behind it, which makes it a sound default when you're building out AI infrastructure.

MCP

The Model Context Protocol (MCP) standardises how language models connect to external tools and data, replacing the custom, per-connection integration work teams would otherwise write by hand. Servers are straightforward to create, and a growing ecosystem of community-built ones cuts the effort further. MCP is now a vendor-neutral project governed by the Agentic AI Foundation under the Linux Foundation, the same body that stewards AGENTS.md.

Uptake has been fast. Some large organisations are aiming to make every internal API AI-accessible through MCP.

Security needs to be a first-class concern. Every MCP server must enforce authentication and authorisation independently of the calling model, with tool grants following the principle of least privilege. Audit logging of all tool calls is essential for traceability. MCP servers returning untrusted data can become an indirect prompt injection vector, so outputs from external sources need careful sanitisation before being fed back into model context.

A broader architectural concern surfaced in April 2026 when OX Security demonstrated that MCP's STDIO transport executes arbitrary commands without validation, leading to multiple CVEs across downstream projects. Their proof of concept also showed malicious entries accepted by most MCP registries. Treat MCP servers sourced from public marketplaces with the same caution as any third-party dependency: review the code, pin versions and run servers with minimal permissions.

For simpler workflows that operate on local files and code rather than external services, Claude Skills offer a lighter-weight alternative worth considering before committing to MCP server development.

We tend to think of MCP as headless plumbing, a way to hand an agent the tools and data it needs with no human in the loop. The surprise this quarter is MCP Apps, an official extension that lets a server render a visual interface, a form or a live dashboard, directly in the host. It is early but not a toy: Claude, VS Code and Microsoft 365 Copilot already support it, and companies such as Postman, Shopify and Hugging Face are building on it. Server-supplied UI opens a fresh attack surface, so treat it as promising rather than proven.

See also: Claude Skills, Agentic tool use.

Trial

Promising technologies with growing adoption, worth exploring for specific use cases.

Microsoft Agent Framework

Microsoft Agent Framework (MAF) launched in October 2025, merging AutoGen's agent abstractions with Semantic Kernel's enterprise features into a single open-source platform. MAF supports Python and .NET, offering graph-based workflows for multi-agent orchestration alongside session-based state management and telemetry.

The framework is still reaching stability, which keeps it in Trial, but it consolidates Microsoft's previously fragmented agent story, and teams building multi-agent systems on Microsoft's stack should evaluate it as their primary framework.

As with any agent framework granting tool access, apply least-privilege scoping and build in prompt injection awareness from the start.

See also: Agentic tool use, MCP, A2A, Vendor agent SDKs.

Vendor agent SDKs

The frontier labs now ship their own SDKs for building agents directly against their models: Anthropic's Claude Agent SDK, OpenAI's Agents SDK and Google's Agent Development Kit. Each hands you the agent loop, tool calling, sessions and context management as a library you drive from your own code, so you write less orchestration yourself.

The Claude Agent SDK runs the harness that powers Claude Code as a library, with its built-in tools, subagents, hooks and permissions. OpenAI's Agents SDK is the production-ready successor to Swarm, code-first by design: your server owns the tools, state and approval decisions while the SDK runs the loop. Google's ADK is the most enterprise-oriented, model-agnostic through LiteLLM though tuned for Gemini, with Vertex AI's Agent Engine as its managed runtime.

There's the standard trade-off against agnostic frameworks such as LangGraph and Microsoft Agent Framework: a vendor SDK tracks its model's newest capabilities closely, at the cost of coupling you tightly to one provider. Use a first-party SDK when you're committed to a model and want its full capability surface, and an agnostic framework when portability is the priority.

All of these run tool-calling agents that act on files, shells, APIs and MCP connectors, so take prompt injection seriously: content returned by a tool can steer the agent into actions you didn't intend. Scope tool permissions tightly, keep a human approval step for anything that changes state, sandbox execution and treat everything a tool returns as untrusted input.

See also: Microsoft Agent Framework, MCP, Agentic tool use, Managed agent platforms.

A2A

Google's Agent2Agent (A2A) protocol standardises communication between AI agents. Launched in April 2025 and now governed by the Linux Foundation, it lets agents from different providers discover each other's capabilities and collaborate without custom integration.

It complements MCP rather than competing with it: MCP connects models to tools and data sources, A2A handles agent-to-agent communication. The design centres on "Agent Cards" that advertise capabilities in JSON, so agents can delegate tasks to one another dynamically, over text or video streaming.

A2A's complexity is justified when agents must collaborate across runtime boundaries, whether different vendors, stacks or organisations. Inside a single runtime, simpler options usually suffice: an orchestrating agent calling tools directly through MCP, a workflow engine such as Temporal or an AI-powered workflow automation platform coordinating LLM steps deterministically, or a multi-agent framework such as LangGraph or Microsoft Agent Framework handling orchestration in-process. Reach for A2A when none of these fit your topology.

LLM testing frameworks

DeepEval evaluates LLM outputs against built-in metrics for relevance, factual accuracy, hallucination detection and toxicity. It integrates with pytest, so teams already running Python test suites can adopt it without new tooling.

Promptfoo takes a CLI-first approach with strong CI/CD integration. Where DeepEval is a Python library you embed in your test suite, Promptfoo runs as a standalone tool comparing outputs across models and prompt variants. OpenAI acquired Promptfoo in March 2026, though it remains open-source under the MIT licence. Teams running multi-model strategies should consider whether the acquisition affects their comfort with Promptfoo as a neutral evaluation tool.

Consider DeepEval if you want evaluation inside an existing Python test suite, and Promptfoo if you want standalone prompt regression testing, particularly where CI/CD pipelines gate deployments on the results.

See also: LLM-as-a-judge, AI red teaming tools.

LangChain & LangGraph

LangChain handles general-purpose LLM interactions while its companion LangGraph extends this to stateful, graph-based agent workflows.

Both reached 1.0 in late 2025, with a commitment to no breaking changes before 2.0. That settles the abstraction churn that made earlier versions awkward to build on. LangGraph's graph model is worth the overhead when you need durable, resumable state across long-running or multi-

agent workflows. It stays in Trial because simpler agents rarely need it: if no state has to persist across a crash or a pause, a provider SDK and a plain tool loop is less to maintain. Try it where your control flow and state are complex enough to justify the framework.

See also: Microsoft Agent Framework, AutoGen.

Formal specification languages

Formal specification languages describe system behaviour precisely enough that properties can be verified before code is written. They sit on a spectrum from lightweight languages that structure intent, through model checkers that explore state spaces, to theorem provers that deliver mathematical proof. TLA+ is the most widely adopted for distributed systems; Amazon used it to find subtle AWS bugs that testing could not surface. Alloy takes a lighter, auto-analysed approach, while Dafny and Lean 4 have drawn substantial AI-assisted proof-synthesis work this year. AI assistants have lowered the barrier enough to make specification viable well beyond the safety-critical systems that once justified it.

We develop Allium, our own language at the practical end: it captures behaviour in a structured, machine-readable form that AI agents use to guide implementation and generate tests. It cannot prove properties across all states the way TLA+ or Alloy can, but it captures intent precisely enough to serve as a contract between human and agent.

AI has shifted the economics of rigour. Agents have rendered formal specification much less costly and much more necessary, given how readily models resolve ambiguity in directions you did not intend.

See also: Spec-driven development, Neurosymbolic AI.

Assess

Emerging or specialised technologies that merit evaluation for specific use cases.

JAX

JAX combines NumPy's familiar API with hardware acceleration and automatic differentiation, and interest in it is growing. TensorFlow and PyTorch remain dominant, but JAX is gaining traction in research settings and among teams working on custom ML architectures.

Its functional approach to ML computation and its ability to compile to multiple hardware targets through XLA (Accelerated Linear Algebra) distinguish it from more established frameworks. It suits projects that need high-performance numerical computing, though teams should weigh its relative immaturity in deployment tooling and a smaller ecosystem of pre-built components. We recommend experimenting with JAX on research projects or contained proofs-of-concept before considering broader adoption.

DSPy

DSPy treats prompts as optimisable programs rather than handcrafted text. You define signatures (input-output specifications) and modules (composable building blocks), and DSPy's optimisers automatically generate effective prompts based on example data. The optimisation process can discover strategies that humans might not have considered.

The framework shows particular promise for complex pipelines involving multiple LLM calls or retrieval steps. DSPy remains in Assess because it introduces a programming paradigm unfamiliar to most teams, and the investment pays off only for systems that benefit from automated prompt optimisation. For simpler single-prompt applications, traditional approaches may remain more practical.

LinkML

LinkML allows teams to define data models in YAML and generates multiple outputs: JSON Schema for validation, Python dataclasses for code, RDF/OWL for semantic web compatibility and documentation. This makes it valuable for phased ontology development where teams want to start practically but preserve the option for formalisation later.

The framework emerged from biomedical informatics but applies broadly. For AI applications, LinkML models can define entities and relationships for knowledge graphs and structured output schemas for LLMs. It remains in Assess because adoption is relatively niche. Organisations already committed to JSON Schema may find less incremental value, but for teams starting fresh on knowledge representation, LinkML offers a middle path between ad-hoc schemas and full OWL modelling.

PydanticAI

PydanticAI brings the developer experience of FastAPI to generative AI application development. It is built by the team behind Pydantic, the validation layer that underpins the OpenAI, Anthropic and Google ADK SDKs alongside LangChain, LlamaIndex and others. It offers model-agnostic LLM support, structured responses through Pydantic validation and a dependency injection system for testing.

PydanticAI reached v1 in September 2025 and has since added durable execution, streaming structured outputs and graph-based control flow. It uses existing Python patterns rather than introducing new paradigms, which makes it immediately approachable for teams already familiar with the ecosystem. Production deployments are appearing, and for Python-first stacks it is a strong default to evaluate.

Prolog

Prolog has regained relevance for neurosymbolic AI architectures. This decades-old logic programming language gives you guaranteed logical inference with explainable reasoning chains, which LLMs can't reliably supply: they handle ambiguous language well but can't follow complex rules or explain why they reached a conclusion. By coupling an LLM with a Prolog reasoning engine, teams can build systems where the model handles the input and Prolog enforces business logic, validates conclusions or traverses knowledge graphs. Implementations typically use Prolog to represent domain rules that validate LLM outputs before they reach users. This pattern suits regulated industries where decisions must be auditable.

We keep Prolog in Assess because the tooling for LLM integration remains immature and performance can suffer at scale. Teams should also consider whether semantic web technologies (RDF, OWL, SPARQL) might serve similar purposes with better tooling support.

See also: Neurosymbolic AI, Ontologies for AI grounding.

Hold

Not recommended for new projects; better alternatives exist.

AutoGen

AutoGen is now in maintenance mode, receiving bug fixes only. In October 2025 Microsoft merged AutoGen and Semantic Kernel into the Microsoft Agent Framework (MAF), which consolidates both projects' capabilities into a single platform. Semantic Kernel is likewise in maintenance mode. Teams currently using either framework should plan their migration to MAF; new projects should start there directly.

TensorFlow

TensorFlow remains a capable deep learning framework and helped popularise machine learning at scale, but its large API surface, fragmented deployment story and intricate architecture make it harder going than more modern alternatives, especially for newcomers.

PyTorch is now the community favourite for research and production, with a more intuitive model and better debugging. For new projects we recommend PyTorch or higher-level tools unless you have a reason to stay on TensorFlow, such as existing deployments or TensorFlow Extended (TFX) for ML pipelines.

Keras

Keras has moved from a standalone deep learning framework to tighter integration with TensorFlow.

Keras was an intuitive entry point into deep learning, with an API that made neural networks more accessible, but the ecosystem has moved on. PyTorch has gained momentum, with clearer debugging, better documentation and a more Pythonic approach. Lightning and FastAI offer similar ease of use while staying closer to current practice. For new projects, we recommend these alternatives rather than investing in Keras-specific expertise.

R

R has a long history in data science and statistical computing, but we place it in Hold for new projects. It remains capable for statistical analysis and visualisation, though its adoption is declining in favour of Python's broader ML and AI ecosystem.

The industry now prefers Python-based ML frameworks, and Python integrates more tightly with modern AI platforms. R retains some advantages for specific statistical applications and academic research, but teams starting new AI initiatives will benefit from standardising on Python and the AI libraries that reach it first.

Techniques

Methodologies and practices for building AI systems: approaches such as RAG, prompt engineering, agent design patterns and evaluation. The "how" of AI development.

Adopt

19. Classical ML
20. RAG
21. LLM-as-a-judge
22. BERT variants
23. Few-shot prompting
24. Agentic tool use
25. Spec-driven development
26. Claude Skills

Trial

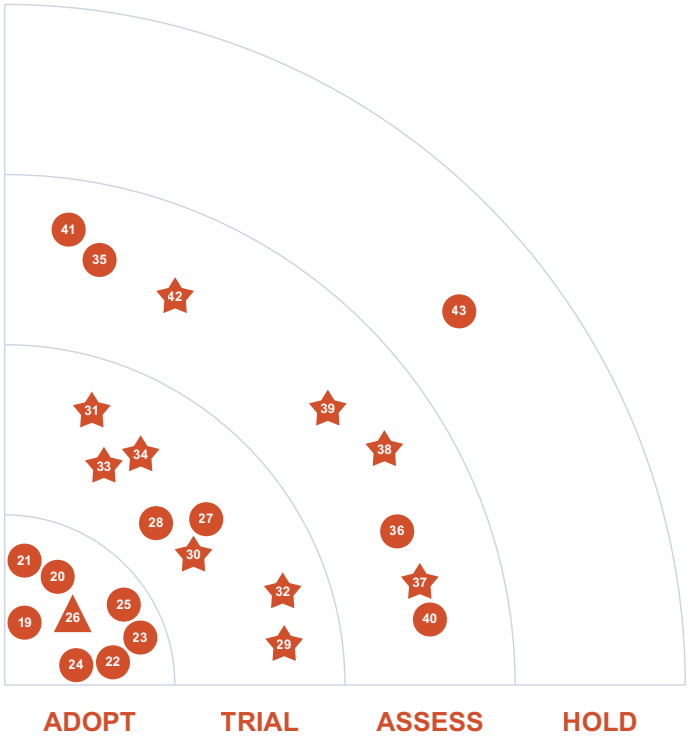
27. Ontologies for AI grounding
28. Structured RAG
29. Orchestrator-worker agents
30. Evals as a release gate
31. Harness engineering
32. Context engineering
33. AI content provenance
34. AGENTS.md and steering files

Assess

35. Neurosymbolic AI
36. LLM reproducibility
37. GEPA
38. Recursive language models
39. Purpose-built classifiers
40. Fine-tuning with LoRA
41. CaMeL
42. Agent identity

Hold

43. AI pull request review



Adopt

Mature, well-supported approaches ready for production use.

Classical ML

Classical machine learning covers approaches such as random forests, gradient boosting (XGBoost, LightGBM), linear and logistic regression and support vector machines. For structured data problems they strike the best balance of explainability and efficiency, routinely outperforming more complex approaches on tabular data while training faster and costing less to run.

The returns depend on quality training data and staff with the right expertise. LLM-based solutions have opened AI to organisations without data science teams, but classical ML still demands specialised knowledge in feature engineering and model selection. If you have those capabilities, these methods work well even on smaller enterprise datasets, matching or exceeding more complex approaches while staying easier to interpret and maintain.

RAG

Retrieval-Augmented Generation (RAG) combines search and text generation to ground responses in real data and reduce confabulation. It's valuable where accuracy and traceability are the priority, such as customer service or compliance. Evaluate your document processing and embedding strategy, and note that RAG introduces an indirect prompt-injection surface: adversarial content in any retrieved document reaches the prompt through the retrieval step, so apply retrieval access controls and track provenance on ingested documents. We're watching techniques such as Self-RAG, which prompts the model to gather more evidence before answering and shows reductions in so-called hallucinations.

RAG is a fundamental technique for grounding agents, but it's not the best solution in every case. For visually complex documents, the usual parse-OCR-chunk-embed route mangles tables, forms and scanned pages. One emerging solution is late-interaction visual retrieval: the ColPali and ColQwen family instead embeds the page image at patch level and matches with MaxSim so layout survives. The cost is a much larger index, but this may be worth the trade-off if your corpus is scans rather than clean prose, and index sizes can be reduced with compression such as MUVERA. RAG isn't always the answer for plain text either. For coding agents, Anthropic's Claude Code team found that direct agentic search, e.g. grep, glob and reading files, beats a pre-built embedding index.

See also: Structured RAG.

LLM-as-a-judge

LLM-as-a-judge uses one model to score another's output, and it's become a practical way to evaluate AI systems. Today's strongest models give nuanced, multidimensional critique that simpler methods can't match, except on very constrained metrics such as exact match or BLEU scores.

The technique is widely adopted in both offline and online evaluation. Offline, it scales far better than human assessment, allowing teams to test thousands of outputs quickly. Online, a judge can score output in real time, letting you adjust the workflow on the quality it reports.

Research demonstrates that frontier models provide judgements correlating strongly with human preferences across many evaluation dimensions. We recommend using a different LLM as the judge than the one being evaluated, and viewing this as an augmentation to human evaluation rather than a replacement. The strongest LLMs can identify nuanced issues in reasoning and factuality that would otherwise require substantial human review time.

See also: Evals as a release gate, LLM observability tools.

BERT variants

Bidirectional Encoder Representations from Transformers (BERT) changed NLP by processing words in relation to their whole context rather than sequentially, and the family keeps evolving, with ModernBERT the latest iteration. Where generative models produce text, BERT-style encoders are optimised for understanding tasks such as classification and sentiment analysis, and they remain the basis for the semantic embeddings RAG uses.

We recommend DeBERTa for new NLP projects, for its disentangled attention; DistilBERT is smaller and faster where latency is tight; and domain-specific variants exist for biomedical (BioBERT) and financial text (FinBERT).

Few-shot prompting

Few-shot prompting gives a model a handful of examples to steer its responses, and it's worked consistently across large language models. The advice is shifting, though.

As models grow more capable, interactive multi-turn approaches are gaining favour: instead of supplying examples upfront, practitioners prompt the model to ask clarifying questions and iterate toward a solution. The pattern often produces better results, particularly in agentic workflows where the model can refine its approach.

Reasoning models shift the balance further. OpenAI and Anthropic both advise defaulting to zero-shot for them and stating explicit success criteria the model can check its output against, rather than supplying worked examples which can even degrade reasoning-model performance. Persona framing has weakened too: research finds that "act as an expert" prompts do not reliably improve factual accuracy, though they still help steer tone and format.

Few-shot prompting retains an important role in non-interactive contexts. System prompts and automated pipelines do not afford clarifying dialogue, and well-chosen examples remain the most effective way to establish output format and domain conventions. We typically see diminishing returns beyond 3-5 examples, with token consumption as the main trade-off.

Agentic tool use

Agentic tool use lets an assistant act as well as talk: edit files, run tests, execute shell commands and perform web searches. Coding assistants that can do this are far more useful than those limited to conversation, which is why we've moved the technique to Adopt for local, sandboxed environments.

The ecosystem has matured to support this. Standards such as MCP and OpenAI's Function Calling provide reliable integration patterns, while improved observability tooling lets teams monitor what agents are doing. The Development Containers specification makes it straightforward to isolate agent execution.

The risk grows once an application accepts external user input, because prompt injection is still unsolved. An agent that safely edits files for a developer becomes a liability when it processes untrusted input. So adopt it for local developer tooling and internal workflows, but proceed with great caution for customer-facing systems, treating each tool permission as a potential attack vector.

See also: Visual computer use agents, Model Context Protocol, Temporal.

Spec-driven development

A natural-language prompt has ambiguities the model resolves silently, so the code an agent returns is sometimes not the code you thought you asked for. Spec-driven development is the industry's response: a written specification becomes the contract the agent builds against, and the plan-implement-verify loop in agentic IDEs works from that spec. Tools such as GitHub Spec Kit and Amazon's Kiro have made the practice mainstream, and the discipline is worth adopting: a reviewed spec gives the agent a clear target, and gives you something to check the result against.

A specification written in prose, though, only moves the ambiguity from the prompt into the spec. Markdown, and even structured forms like EARS or Gherkin, still rely on the model to interpret the words. Nothing stops two requirements contradicting each other, and nothing checks that the spec is consistent before the agent builds on it.

We are following the rising popularity of formal specification languages, though the field is young and adoption is still limited. Because a formal specification is machine-readable, you can test it for contradictions and mutually exclusive requirements before any code is written. That doesn't make the spec correct: even a consistent one can describe behaviour you didn't actually want.

The case is stronger still when the specification is durable, something you maintain alongside the code as it changes rather than write once and discard. A formal language with well-defined semantics is much easier to update than prose, where each edit can let ambiguity back in. Our own Allium is one such language: it gives the agent an unambiguous reference, and it flags conflicting requirements within the spec. For regulated teams, who must show their requirements are consistent and their code meets them, it is worth investing in now.

See also: Formal specification languages, Claude Skills.

Claude Skills

Claude Skills are self-contained folders, a SKILL.md of instructions with optional scripts and reference files. They package a procedure the assistant loads on demand, so you don't re-explain or paste it in each session. Our teams use them daily for drafting proposals, structured debugging, commit messages and PR descriptions. The format now works across several coding assistants, so it doesn't lock you to one vendor.

For many jobs a Skill is simpler than an MCP server, which needs a running server and a protocol to maintain. A Skill has neither, so it fits best when the work stays inside your own project. Use MCP when the assistant needs to reach a running service beyond it.

An installed Skill can run code, with the usual supply-chain and data-exfiltration risks. The exposure depends on the host: in Claude Code a Skill has full access to the local machine and network, while the API runs it in a sandboxed container with no network by default. Treat a third-party Skill like any other dependency: review it before you install it, and prefer first-party or self-authored ones. Keep the agent's access tight while one runs.

We place Skills in Adopt because the one caveat, running code you didn't write, is the same supply-chain risk as any other dependency, kept in check by the review and least-privilege habits above.

See also: MCP, Agentic tool use, Spec-driven development, Harness engineering.

Trial

Promising approaches with growing adoption, worth exploring for specific use cases.

Ontologies for AI grounding

An ontology is an agreed, explicit model of the entities in your domain and how they relate. An LLM already has a rich, learned sense of those concepts from training, but that understanding is implicit and outside your control: you can't inspect it or make it match the definitions your business and regulators rely on. Grounding retrieval and agent output in an ontology gives you definitions you set and can audit, and lets the system follow relationships through a knowledge graph to answer questions that plain RAG can't.

Ontology work usually fails in one of two ways: a formal route that pursues completeness in OWL and never ships, or an ad hoc one that starts as a spreadsheet and grows unmaintainable. Start small and formalise only where you need it, and the tooling is mature. Graph databases such as Neo4j store the model, and LinkML lets you define it in YAML without deep ontology expertise. Begin with one painful, high-value domain, not the whole organisation.

If you find ontologies too constraining, semantic spacetime classifies relationships by the role they play rather than fixing them in a schema. We're watching this direction.

See also: LinkML, Neurosymbolic AI, Prolog.

Structured RAG

Structured RAG extends basic RAG by organising retrieved knowledge as graphs, schemas or typed records rather than flat text chunks. Microsoft's GraphRAG uses an LLM to build a knowledge graph from source documents during indexing, then queries that graph at retrieval time. This addresses a weakness in standard RAG: questions requiring synthesis across many documents, not a single passage.

GraphRAG has matured since our last radar. LazyGraphRAG reduced indexing costs to a fraction of the original, removing the biggest barrier to adoption. Neo4j provides examples combining graph-based retrieval with LLM generation.

The trade-off remains upfront investment. Graph-based indexing requires more compute and design than vector-based RAG, and the knowledge graph must be maintained as source documents change. If your queries are primarily about finding relevant passages, standard RAG with cross-encoder reranking may suffice. If they do require reasoning across documents, structured approaches justify their cost.

See also: RAG, Ontologies for AI grounding, Cross-encoder reranking.

Orchestrator-worker agents

When a task is too big for one agent, or breaks into independent parts, you can spread it across several agents. In the orchestrator-worker pattern, one orchestrator drives the overall task, delegating self-contained subtasks to workers, each with its own context window, and gathering their results. Anthropic's multi-agent research describes this hierarchy, now built into tools like Claude Code's subagents.

Only one agent should make changes. Route decisions and edits through a single writer and have the others only read and report, because several agents acting in parallel make conflicting changes that someone has to unpick, which is the core of Cognition's case against multi-agent systems. Keeping to one writer also limits a security risk. A worker that reads external data can pick up a prompt injection, so it shouldn't be acting as well: when workers only read and report, their output can be treated as untrusted, and the single writer decides what to act on.

It's expensive, because each worker runs its own context and the token cost multiplies, and is a poor fit for tightly interdependent work like writing code, where the parts can't be cleanly separated. The pattern works best on tasks that split into independent pieces, such as parallel research or a breadth-first search across a large codebase.

See also: Agentic tool use, Spec-driven development, Agent memory architectures, CaMeL.

Evals as a release gate

An AI system has no compiler to catch a regression. A reworded prompt, a model upgrade or a new tool can make the output worse, and often the first signal is rising user complaints. The defence teams have settled on is to treat evaluations as regression tests: score a fixed set of cases on every change and block release when a score drops below its threshold or the last known-good baseline. As Simon Willison puts it, evals are regression tests for non-deterministic systems.

What you gate on depends on the system: task accuracy against a labelled set, cost, latency and jailbreak resistance for most, with agents also checked on whether they chose the right tool and kept to the plan, and retrieval traced back to source. Where a judgement is too nuanced for a simple assertion, LLM-as-a-judge does the work. Promptfoo, DeepEval and Langfuse all run these gates in CI.

We put it in Trial because the gate pays for itself wherever a regression would be costly in production, and for regulated teams it gives model risk management an auditable quality bar. Whether to write the evals first, test-driven style, is contested: Hamel Husain argues for writing evaluators against the failures you've already seen rather than ones you imagine up front. Begin with the few cases you know are critical and let the suite grow from the incidents you hit.

See also: LLM-as-a-judge, LLM testing frameworks, LLM observability tools, AI red teaming tools, Production AI monitoring platforms.

Harness engineering

An agent is a model plus a harness, the code around the model that assembles its context, manages its tools, runs its loop, keeps its memory and checks its work. As frontier models converge, the harness increasingly decides whether an agent is reliable: a capable model flails in a poor harness, and a modest one does useful work in a good one.

Much of the craft is removing things and adding structure. When Vercel cut most of an agent's tools, accuracy rose: a smaller toolset gives it less room to go wrong. The rest is choosing the right loop, setting a stop condition the model can't just declare met, and resetting context on long jobs so quality doesn't decay. You don't have to write all this yourself: the Claude Agent SDK and OpenAI Agents SDK give you a ready loop, tool handling and context management.

The harness is also the control point for security and audit. A leaner toolset is a smaller attack surface, and least-privilege scoping limits the damage from a prompt injection; tool scope, data provenance and a captured trace all live here, which is what model risk management needs. We put harness engineering in Trial: the vocabulary is only months old and the tooling and measurement still settling, but the practices pay off today and we use them ourselves. Treat the harness, not the model, as the thing you're building.

See also: Agentic tool use, Orchestrator-worker agents, Spec-driven development, AGENTS.md and steering files, Agent memory architectures, Context engineering.

Context engineering

A long-running agent rarely fails by running out of context window; it fails because the window fills with noise and, as tokens climb, recall and judgement fall away, the effect Anthropic calls context rot. Deciding what stays, and what is summarised, offloaded or dropped, is now a discipline of its own.

It sits next to harness engineering: the harness is the scaffolding around the agent, context engineering is the token budget within a single run. A few techniques help: compaction summarises near the limit and restarts; structured note-taking offloads state to a file; sub-agent isolation hands a subtask a clean window; and tool-result clearing replaces spent outputs with placeholders. Anthropic now exposes compaction, tool-result clearing and a memory tool as API primitives.

For regulated teams, compaction keeps a run going by discarding detail: it preserves the high-level thread, but specific figures and the step-by-step trail can be lost, with nothing in the summary to show it. If an agent's reasoning has to be reconstructed for model risk management or audit, keep the raw trace. We place context engineering in Trial: the primitives are young and in beta, and the patterns for compacting rather than offloading are unsettled, but the problem is universal for anyone running agents past one window, and the mitigations exist today.

See also: Harness engineering, Agent memory architectures, Orchestrator-worker agents.

AI content provenance

Marking AI-generated content is now a legal requirement. Under Article 50 of the EU AI Act, transparency duties apply from 2 August 2026: a system generating synthetic audio, images, video or text must mark its output in machine-readable form, a chatbot must tell users they're talking to a machine, and text published to inform the public must be labelled. The duty falls on providers and deployers, and a breach sits in the middle penalty tier, up to €15M or 3% of global turnover.

The Act doesn't say which technology to use. The European Commission's voluntary Code of Practice points to two layers: signed metadata, and invisible watermarking for when it's lost. In practice the metadata layer means C2PA Content Credentials, the signed-provenance standard backed by Adobe, Microsoft, OpenAI and the BBC.

We place this in Trial because the legal driver is live and the tooling is usable today, but neither layer is dependable on its own. C2PA metadata is easily stripped: a screenshot or re-encode breaks the chain, and it rarely survives across platforms. Watermarks survive edits better but remain imperfect; Google's SynthID is probabilistic and removable, and 2026 research has stripped and forged such marks. Check each pipeline on its own terms rather than assuming one mechanism meets the obligation.

See also: Foundation models, AI governance platforms.

AGENTS.md and steering files

AGENTS.md is a plain markdown file that tells a coding agent how to work in a repository: how to build and test, the target architecture and conventions to follow, and the things to leave alone. It began as an OpenAI Codex convention and is now a vendor-neutral standard stewarded by the Agentic AI Foundation under the Linux Foundation. Cursor, Codex, Gemini CLI and Copilot read it natively; Claude Code only reads it as a fallback when a project has no CLAUDE.md.

It's part of a wider practice of project-level steering: keeping the context an agent needs in version-controlled files, not an isolated prompt. Larger projects tend to split these by concern (product, architecture, code style) and reference them from the root file. These files must be correct: if an agent generates the first versions, check them closely, since errors hamper every future session.

Adoption is broad and the cost of starting is low, but we place it in Trial because conventions for structuring and sharing them across teams are still forming. It pairs naturally with harness engineering and spec-driven development, which supply the intent behind these files.

See also: Harness engineering, Spec-driven development, Claude Skills, MCP.

Assess

Emerging or specialised approaches that merit evaluation for specific use cases.

Neurosymbolic AI

Neurosymbolic AI combines neural networks with symbolic reasoning to address fundamental limitations of pure LLM approaches. Neural networks excel at pattern recognition and handling ambiguity, while symbolic AI provides logical reasoning and explainable inference. LLMs understand natural language well but cannot guarantee rule compliance or explain their reasoning in auditable ways.

Fundamentally, this is an architectural problem. LLMs operate through probabilistic pattern matching over language, not causal modelling. As Mark Burgess argues in his work on semantic spacetime, language models "paraphrase intentional knowledge" rather than tracing actual causal chains. Precise answers to precise questions require systems that explicitly encode what causes what.

The case is strongest in regulated sectors. Regulatory rules are non-negotiable constraints, not suggestions a model can approximate. Risk models need to know what entities are and how they relate, and compliance requires explainable decision trails. Similar pressures apply across financial services, healthcare and insurance.

Implementations range from lightweight to sophisticated. On the simpler end, teams constrain LLM outputs to ontology terms or use knowledge graphs to ground RAG retrieval. More advanced implementations use symbolic reasoning engines to validate LLM-generated conclusions. Renewed interest in Prolog reflects exploration of logic programming alongside LLMs.

We place Neurosymbolic AI in Assess because production patterns are still emerging, but teams in regulated sectors should be experimenting now.

See also: Prolog, Ontologies for AI grounding, Agentic tool use, World models.

LLM reproducibility

Large language models are non-deterministic even at temperature zero, which is a problem for regulated industries where Model Risk Management frameworks require reproducible, auditable decision-making. Banking regulations such as OCC/SR 11-7 assume a level of model stability that generative AI doesn't provide.

The cause runs deeper than floating-point arithmetic. Research shows that batch-dependent kernel operations make outputs vary with server load, not input alone. Smaller open weight models on controlled infrastructure tend to achieve more reproducible outputs than larger models served via shared APIs. Where stochastic behaviour is acceptable, the variation must be

well-characterised so it can be explained to regulators as a designed property rather than an infrastructure artefact. Prompts and model versions should be treated as versioned code with change control and rollback procedures.

For teams requiring determinism from larger models, SGLang now offers deterministic inference building on batch-invariant operators, with the underlying research selected for oral presentation at NeurIPS 2025. Teams subject to MRM requirements should be actively evaluating their options now.

See also: Neurosymbolic AI, LLM-as-a-judge.

GEPA

GEPA (Genetic-Pareto) optimises the prompts in an AI system automatically. It runs the system, reads the traces (reasoning, tool calls, errors) and asks an LLM to diagnose the failures in plain language and propose prompt edits, evolving a Pareto frontier of candidates that do well on different parts of the task. The paper, accepted at ICLR 2026, reports that it outperforms reinforcement-learning tuning while using far fewer rollouts, and improves on the previous best prompt optimiser: MIPROv2.

The appeal for regulated teams is that GEPA works on API-only models without touching weights, and it produces prompts a human can read and review rather than an opaque set of fine-tuned parameters. It depends on a measurable objective, so it fits tasks with a clear evaluation signal better than open-ended generation. We've placed GEPA in Assess. It is already available through DSPy as `dspy.GEPA`, but it is early, and the quality of the optimised prompt depends heavily on the evaluation metric you give it. Teams already investing in prompt optimisation should be trying it now.

See also: DSPy, Spec-driven development, LLM-as-a-judge.

Recursive language models

Recursive language models (RLMs), from a team at MIT CSAIL, address a familiar problem: a model's output degrades as its prompt grows, well before the context window is technically full. An RLM keeps the long context as a variable in a code environment and gives the model only the query, so the whole context never enters the model's window at once; the model writes code to slice and filter it, calling itself or cheaper sub-models over the parts it needs. Alex Zhang's write-up reports working over context far larger than any single model's window this way. It extends the code-execution pattern (CodeAct) with recursive calls, and where orchestrator-worker agents split a task along a plan, an RLM splits the context itself and lets the model decide how.

Because the model writes and runs code over context that's often untrusted, a malicious document can steer the generated code, so sandbox the REPL with least privilege: no network and no filesystem reach beyond the context variable itself. We place RLMs in Assess. The

research is credible and there's an active open-source implementation, but the results are largely author-reported, the latency is unpredictable and we've seen no production use. Try it where context length is the binding constraint.

See also: Agentic tool use, Orchestrator-worker agents, RAG.

Purpose-built classifiers

Many of the jobs teams give an LLM are classification tasks: routing a request, detecting intent, scoring a document or deciding whether an action is safe. A model built for one of these answers in milliseconds, costs a fraction of a frontier API call, and returns a label you can act on, often with a calibrated confidence score. Place one in front of an agent to handle the straightforward cases, and the expensive model gets called far less often.

Encoder classifiers have done this work for years. GLiNER and GLiClass classify against arbitrary labels with no task-specific training, run on your own hardware, and come from the same BERT family already in our Adopt ring. A small encoder fine-tuned on a few thousand of your own examples will usually beat a hosted service on cost and latency, and it keeps the data in house.

Most of the interest this quarter is in TypeSafe's Jev, a proprietary hosted model that returns a calibrated distribution over a fixed set of answers, through its own API or OpenRouter's experimental decisions endpoint. Independent testing confirms the speed and the price, though the claim that it can't hallucinate is overstated and its headline benchmarks look stronger than the detailed results. We put the category in Assess: the approach is sound and worth a place in the toolbox, but there's little reason to use a closed service for work that open encoder models already do on infrastructure you control. Start with the mature open options and treat the hosted newcomers as a convenience, not a default.

See also: BERT variants, Small language models, Evals as a release gate.

Fine-tuning with LoRA

Low-Rank Adaptation (LoRA) fine-tunes a model by training a small set of added parameters and leaving the base weights frozen. That keeps training cheap enough to run on modest hardware and produces a lightweight adapter rather than a full copy of the model, close to the quality of a full fine-tune. It has made customising open-weight models practical for teams without large training budgets.

Tools such as LitGPT and axolotl make the training loop straightforward, but applying LoRA well still takes ML expertise and clean, well-chosen training data. The larger risk is lock-in: a fine-tune ties you to one base model, and models improve quickly enough that a newer general-purpose release can overtake the one you tuned months ago, leaving you to maintain and re-tune it. We place it in Assess rather than Trial, and would reach for it only where the business value clearly outweighs that ongoing cost.

Reinforcement fine-tuning is the other route to a tuned model. LoRA learns from examples; reinforcement fine-tuning learns from a reward: run the model on a task, score the output with a programmable grader or an LLM judge, and apply GRPO-style updates that raise the odds of the behaviour that scored well. Through 2026 it became a common way to turn an off-the-shelf model into a reliable agent for tool-calling and coding, and a tooling layer grew up around it: RL environments packaged as installable Python packages such as Prime Intellect's verifiers and OpenPipe's ART, with LLM-as-judge reward shaping such as RULER. It sits in Assess alongside LoRA: for most teams reward design and environment construction are still specialist work, the failure mode is reward hacking rather than the behaviour you were after, and an RL-tuned model is harder to account for under model risk management than a prompted one, because its behaviour lives in shifted weights rather than an inspectable prompt.

CaMeL

CaMeL (CApabilities for MachinE Learning) is a defence architecture from Google DeepMind for mitigating prompt injection in agentic systems. Its paper, *Defeating Prompt Injections by Design*, treats prompt injection structurally rather than through prompt cleverness or red-teaming. The architecture splits responsibilities across two models: a privileged P-model processes only user instructions and emits a program of execution steps, while a quarantined Q-model handles external data but cannot call tools directly. A custom interpreter tracks data provenance, enforcing capability-based security so untrusted data cannot escalate privileges. The implementation is open source.

Prompt injection is the dominant unsolved problem for agentic systems, and it recurs throughout this radar, in MCP, agentic tool use, AI red teaming tools and our Hold placement for OpenClaw. Most defences are statistical, such as filters, classifiers and harnesses. CaMeL is one of the first to make the architecture itself prove that untrusted data can't reach privileged operations, reducing many attacks on the AgentDojo benchmark to zero.

We place CaMeL in Assess: it addresses prompt injection more rigorously than anything else we have seen, but production patterns have not emerged, users must author security policies, and running two models adds latency and cost. For teams building agentic systems in regulated industries, the paper is required reading.

See also: Agentic tool use, OpenClaw.

Agent identity

When an agent acts, whose authority is it acting under? Most production systems cannot answer cleanly. Agents read and write through a shared service account or a long-lived API key, so there is no record of which agent took an action, let alone which human authorised it. Over-permissioned, shared credentials are among the most common and hardest-to-retrofit failures in agent deployments, and they are exactly what least privilege and clear trust boundaries exist to prevent.

The fix reuses established non-human identity practice. Give each agent its own scoped, short-lived credential instead of a shared secret. SPIFFE issues cryptographically attested workload identities on this model, OAuth 2.1 with PKCE covers browser-driven agents, and signed delegation tokens preserve the chain of authority, from the human who triggered the work through each agent that acted on it. Bind every action to that identity, so you can trace who did what and revoke the credential when the agent is retired.

Attention is catching up to the risk. OWASP's Top 10 for Agentic Applications puts identity and privilege abuse among its leading risks, and NIST opened an AI agent identity and authorisation initiative in early 2026. For regulated teams the driver is sharper still: HIPAA's unique-user-identification rule, GDPR's accountability principle and SOX audit trails all assume you can say who did what, which a shared agent account cannot support.

We place this in Assess. The underlying primitives are production-proven outside AI, but the agent-specific patterns and tooling are young and still settling, so experiment now rather than retrofit attribution later.

See also: Agentic tool use, MCP gateways, CaMeL, Agent memory architectures.

Hold

Not recommended for new projects; better alternatives exist.

AI pull request review

AI now reviews code well. Developers who work effectively with multi-turn conversations can get useful feedback at every level, from syntax issues and architectural patterns to subtle runtime concerns such as race conditions.

We keep AI pull request review in Hold for organisational rather than technical reasons. In addition to finding errors, PR review also shares knowledge: senior developers mentor juniors, and the team keeps track of how the codebase evolves. Teams who delegate review to AI often see a decline in collective code ownership.

We recommend using AI as a first-pass reviewer to catch issues before human review, but preserving the human step as deliberate practice for team alignment and knowledge transfer.

Tools

Software that enhances AI development workflows without being embedded in your application code: IDE extensions, CLI utilities, testing frameworks and observability.

Adopt

44. Software engineering copilots
45. Provider-agnostic LLM facades
46. Notebooks

Trial

47. Vector databases
48. Local model execution environments
49. LLM observability tools
50. LLM API gateways
51. AI red teaming tools
52. AI-assisted code migration
53. Agent code sandboxes

Assess

54. Supervising multiple agents
55. Structured output libraries
56. MCP gateways
57. Agentic vulnerability discovery
58. AI supply-chain security

Hold

59. OpenClaw



Adopt

Mature, well-supported tools ready for production use.

Software engineering copilots

Copilots that write and run code from a natural-language brief are now part of normal engineering practice, which is why they sit in Adopt. Most teams already have one in place, so the practical choices now are which tool to pick and how best to work with it.

Broadly, the tools split into two kinds: those you can point at any provider, and those tuned to a single model. Among the model-agnostic ones, OpenCode stands out for its terminal experience and breadth of integration; Cursor, Windsurf and Zed are standalone editors; and CLI tools such as Aider, Cline and Block's Goose run against any provider. The provider-specific tools, Claude Code, Gemini CLI and OpenAI Codex, are each built around their own model, while GitHub Copilot and Tabnine live inside the IDE.

The field keeps fragmenting, with no consolidation in sight. Claude Code is the strongest for long, autonomous work but also the heaviest and most tied to one provider, and as usage-based pricing bites, teams increasingly pair a capable proprietary tool for the hard problems with cheaper, model-agnostic ones running on open weight models for routine changes. Most of the recent movement is in that agnostic layer, because it keeps the provider choice open.

Teams work with these tools interactively in two broad styles: free-form "vibe coding", steered by conversation, and a more structured, spec-first approach that works out what's needed before any code is written. Kiro does both, pairing a conversational mode with a specs mode for drafting requirements first. Either way, project conventions increasingly live in AGENTS.md and steering files that every tool can read, rather than in a separate dotfile for each tool.

A newer mode is different again: it hands a scoped task to a cloud agent that runs unattended in its own sandbox and opens a pull request; OpenAI's Codex cloud and Google's Jules both work this way. This shifts the load onto review and widens the attack surface, so scope these agents tightly and keep a person in the loop. We cover the mechanics under Agent code sandboxes and Supervising multiple agents.

Senior engineers get the most from these tools, handing over routine work while keeping a close eye on quality; less experienced developers can struggle to tell a sound suggestion from a plausible but wrong one. The teams that do well treat that as a training problem and keep a trust-but-verify habit.

Provider-agnostic LLM facades

The LLM market moves quickly enough that today's best choice may be displaced within months. A facade between your application and the underlying provider keeps that switching cost manageable. Options range from the lightweight AISuite and Simon Willison's LLM library to heavier alternatives such as LangChain and LlamaIndex. A thin in-house wrapper is also a reasonable choice for teams that prefer to minimise dependencies. We've seen enough projects hampered by tight coupling to a single provider to treat some form of abstraction as a default.

Notebooks

Notebooks remain the de facto standard for data science and ML experimentation. Code, prose and visualisations live alongside each other, which suits the iterative work of model development and gives technical and non-technical collaborators something concrete to discuss. Jupyter is the most widely used, with cloud-hosted equivalents from Google Colab, AWS Sagemaker, Azure and Databricks. Language-specific alternatives include Pluto.jl for Julia, Clerk for Clojure and Polynote for Scala.

Trial

Promising tools with growing adoption, worth exploring for specific use cases.

Vector databases

Vector databases have emerged as specialised tools for managing the high-dimensional embeddings required by AI models. Prominent solutions include Pinecone, Qdrant, Milvus and Weaviate.

Traditional databases may suffice for simpler operations, and alternative approaches such as pgvector with pgvectorscale bring vector search directly into Postgres, avoiding the data consistency challenges of keeping embeddings synchronised across databases. If a dedicated vector database is required, Pinecone leads in production readiness but comes with managed service costs, while open-source alternatives such as Qdrant and Milvus offer greater control but demand more operational expertise.

For prototyping, Chroma offers a Python-first approach with minimal configuration. A 2025 Rust rewrite improved performance, though it remains best suited for small-to-medium scale applications. LanceDB takes a different approach as an embedded vector database, similar in philosophy to SQLite. It operates as a library within your application using Apache Arrow's columnar format, making it compelling for local AI assistants and edge deployments where data must remain on-device. The trade-off is limited high-concurrency support.

One caution on scope. For retrieval over document corpora, where questions and source text use different words for the same idea, vector search remains the right tool. For agents working across a codebase, it increasingly is not: teams are finding that agentic search, direct grep and file traversal, beats a pre-built embedding index that goes stale as the code changes. Reach for a vector database when semantic similarity is the problem, not as a reflex for every retrieval task. See RAG for the fuller argument.

Local model execution environments

Tools such as Ollama, LM Studio and AnythingLLM provide accessible ways to run open weight models on local hardware. These enable experimentation with models from Meta, Mistral, DeepSeek, Alibaba and OpenAI without API costs or sending data to external services. Many now support tool calling via MCP and connections to commercial APIs for hybrid workflows.

These tools serve developers testing AI features, teams comparing model responses and organisations exploring capabilities with sensitive data that cannot leave their infrastructure.

LLM observability tools

Modern agentic builds involve multi-step reasoning, tool orchestration, RAG retrieval and chains of LLM calls where a single user request might trigger dozens of internal operations. Debugging why an agent produced an unexpected result requires visibility into every step of that chain. This is distinct from production AI monitoring, which focuses on drift detection in deployed systems.

Phoenix, from Arize AI, has emerged as a leading open-source option. Built on OpenTelemetry, it provides tracing and evaluation with auto-instrumentation for LangChain, LlamaIndex, DSPy and direct integrations with OpenAI, Anthropic and AWS Bedrock. Langfuse is the most popular fully open-source alternative (MIT licence), combining tracing and evaluation with strong multi-turn conversation support.

For LangChain-committed teams, LangSmith provides native integration that surfaces framework internals in debugging views. Helicone takes a lightweight proxy approach: route API calls through its endpoint for observability without SDK changes. MLflow 3 also offers OpenTelemetry-based tracing for LLMs and agents, a natural choice where a team wants a single platform across classical ML and GenAI. Since these tools capture prompt and response data, data sovereignty is a concern: Phoenix and Langfuse both offer self-hosting for teams with data residency requirements. Both have also added evaluations that run as CI gates, so a tracing tool can now block a regression before it ships; that practice is covered under Evals as a release gate. Galileo uses purpose-built small models (its Luna family) for low-latency evaluation rather than frontier-model judges; Cisco acquired it in 2026, and it now ships as Splunk Agent Observability.

LLM API gateways

As organisations adopt multiple model providers, an infrastructure layer emerges between applications and providers. LLM API gateways handle routing, caching, failover, rate limiting and cost tracking at the proxy level, complementing code-level abstraction from libraries such as AISuite. A facade is a developer choice about how to call models; a gateway is a platform decision about how to manage model traffic across the organisation.

LiteLLM is the most widely adopted open-source option, providing an OpenAI-compatible proxy with spend tracking, budget controls and key management. Portkey offers a managed alternative with semantic caching and conditional routing. Kong brings enterprise API management experience to LLM traffic.

For multi-provider deployments, a gateway centralises audit logging, per-team budget enforcement and automatic failover. Check whether it simplifies the operational story before you build equivalent functionality in-house.

See also: Provider-agnostic LLM facades, LLM observability tools, MCP gateways.

AI red teaming tools

Our security coverage has grown this quarter, with guidance on prompt injection in MCP and RAG and architectural defences such as CaMeL. Red teaming is offensive testing: tools that systematically probe AI systems for vulnerabilities before attackers do.

Promptfoo is the standout, evolving from a prompt evaluation CLI into a comprehensive red teaming platform. OpenAI acquired it in March 2026 but the MIT licence remains. It ships financial services plugins for PCI DSS and banking regulation testing. Microsoft's PyRIT focuses on multi-turn attack orchestration with Azure AI Foundry integration. NVIDIA's Garak takes an agentic approach, autonomously probing for prompt injection, data leakage and toxicity.

The EU AI Act will require adversarial testing for high-risk systems, an obligation the Digital Omnibus moved to December 2027. We recommend starting with Promptfoo for breadth and CI/CD integration.

See also: CaMeL, Agentic tool use, LLM observability tools, Evals as a release gate.

AI-assisted code migration

Large-scale code migration sits in a gap between what copilots do (assist line-by-line) and what bootstrappers do (generate new projects). Migration tools operate at codebase scale, applying thousands of coordinated changes to upgrade language versions, swap frameworks or modernise APIs. The pattern that works best combines deterministic code transformations with AI for edge cases that rules alone cannot handle.

Moderne is the leading platform, built on the open-source OpenRewrite engine. OpenRewrite provides deterministic "recipes" for common transformations such as Java version upgrades, Spring Boot migrations and Jakarta EE transitions, while Moderne adds AI-assisted handling of non-standard patterns and enterprise-scale orchestration.

These migrations are well-understood but labour-intensive, exactly where deterministic transformation augmented by AI pays off. We recommend trialling OpenRewrite on a representative repository before committing to the Moderne platform.

See also: Software engineering copilots, Spec-driven development.

Agent code sandboxes

Agents increasingly write code and run it, to analyse data or drive a tool as they work through a task. That code is untrusted: it's generated at runtime and often shaped by input an attacker can influence, so you don't want it running on the host that holds your credentials and source. An agent code sandbox gives it somewhere safe to run: an isolated, disposable environment with limited filesystem and network access, torn down after use.

The category has purpose-built providers. E2B runs each session in a Firecracker microVM, while Modal isolates sandboxes with gVisor by default and offers a full-VM option for workloads that need their own kernel; Daytona and Cloudflare's Sandbox SDK offer isolated containers with fast start-up; Pydantic's monty is a minimal secure Python interpreter for LLM-generated code with no container overhead. For local work, Anthropic's sandbox-runtime confines an agent's processes at the OS level using Seatbelt, bubblewrap and network namespaces. This is a different concern from local model execution environments, which run the model itself; here we isolate the code the model produces.

We place agent code sandboxes in Trial: the tooling is mature enough to adopt now, and a team that lets an agent run generated code outside a throwaway environment is taking an avoidable risk. Prompt injection makes isolation more important, because the untrusted input flows straight into the code the agent runs. A sandbox contains the damage when that happens; it doesn't stop the injection itself.

See also: Agentic tool use, CaMeL, Managed agent platforms, Supervising multiple agents.

Assess

Emerging or specialised tools that merit evaluation for specific use cases.

Supervising multiple agents

Running several coding agents at once is much harder than running one. You lose track of what each is doing and which one is waiting for you. The field is early, and tools are trying quite different interfaces for it.

Staying engaged. The most useful thing one of these tools could do is work like an inbox: interrupt you only when an agent needs a decision, instead of leaving you to check each one in turn. Few do this yet. Claude Squad shows a list of running agents for you to look through. Conductor and Imbue's Sculptor are built around reviewing each agent's changes, and Sculptor can drop you straight into an agent's work on your own machine to take over. VS Code and GitHub's Agent HQ add a shared view across your agents. None yet lets you move smoothly between agents, and the inbox idea has barely reached coding tools so far. The risk is that you stop reading the changes and just approve them.

Keeping it local. To run agents in parallel, each needs its own copy of the code so they don't overwrite each other: a git worktree per agent, or a container per agent, as Sculptor does. A container can also act as a sandbox, stopping the agent from reaching the rest of your machine or the network. Keep the agents on your own machine and your code and credentials stay with you, or use hosted services such as Factory and Devin, which run them on their own servers and suit sensitive or regulated work less well.

We place this in Assess: the tools change fast, and some have already shut down or gone commercial, so don't become too invested in any one tool. Running each agent in its own workspace is sensible now, and we would prefer tools that keep everything on your own machines.

See also: Agent code sandboxes, Orchestrator-worker agents, Harness engineering.

Structured output libraries

Libraries such as Instructor, Outlines and Marvin bridge a gap: LLMs produce freeform text, but applications need structured data. These libraries constrain outputs to match specified structures through prompting, logit manipulation or grammar-based generation. Instead of hoping an LLM produces valid JSON, developers specify Pydantic models and receive guaranteed-valid objects. For agentic systems this is essential, as agents need to produce function calls and decision objects that downstream code can reliably process.

Before reaching for one, check whether the task needs generation at all. If all you need is one label from a fixed set, that's classification, and a purpose-built classifier such as TypeSafe's Jev returns it directly, faster and cheaper than constraining a generative model to produce it. Keep these libraries for when you need the model to fill in a richer object.

The space is evolving quickly. Instructor has gained traction for its simplicity and Pydantic integration, while Outlines offers more sophisticated constrained generation. Native structured output features from model providers (OpenAI's JSON mode, Anthropic's tool use) may reduce the need for external libraries in some scenarios.

A broader category of runtime guardrails has grown up alongside these libraries. NVIDIA's NeMo Guardrails and Guardrails AI go beyond schema conformance to include prompt injection scanning, content filtering and hallucination checks. Lakera Guard scans inputs and outputs for prompt injection, toxic content and data leakage; following its acquisition by Check Point it is best assessed as part of that stack rather than as a standalone product. Teams building production LLM applications should evaluate both levels: structured outputs for type safety and guardrails for content safety.

See also: PydanticAI, AI red teaming tools.

MCP gateways

An MCP gateway sits between AI clients and MCP servers as a controlled proxy. As MCP moves into production, it raises the same governance questions as any other integration point: who may call which server, with whose credentials, and how it's all audited. A gateway answers them at that one point, adding access control, credential handling and audit logging, and some products also inspect the traffic for data loss and prompt injection.

Several vendors now offer products under this label. Lasso's MCP Gateway concentrates on inspecting content, masking secrets, redacting personal data and blocking malicious servers. MintMCP and Workato add single sign-on, role-based access and identity-aware policy. Docker's MCP gateway runs servers in isolated containers behind one managed endpoint, while Kong brings its existing API-management machinery to MCP traffic. The recurring hard part is credentials: the MCP authorization spec makes each server an OAuth resource server and forbids passing tokens straight through, so most of the engineering effort goes into handling that cleanly at runtime.

We place MCP gateways in Assess. The problem is worth solving and the security concerns are well founded, but the products are young and fragmented, and they differ enough that you can't yet count on moving between them. Pilot one where you're taking MCP into production, alongside the security guidance in the MCP entry. It's a different layer from LLM API gateways, which govern model-provider traffic rather than tool calls.

See also: MCP, LLM API gateways, AI red teaming tools.

Agentic vulnerability discovery

Agents that read source code can now find security vulnerabilities and often propose a fix. Because the agent reasons over the code rather than matching patterns, it catches classes of bug that scanners and fuzzers miss, and it can carry a finding through to a candidate fix rather than stopping at a ticket. This is separate from AI red teaming tools, which probe a model's behaviour for jailbreaks and unsafe outputs; here the target is defects in conventional code.

This is already working in practice. Google's Big Sleep, a Project Zero and DeepMind effort, has found live bugs in widely used code including a SQLite flaw, and Mozilla's run of a frontier model against the Firefox source surfaced logic bugs conventional tooling had missed. GitHub's Copilot Autofix suggests fixes for code-scanning alerts, with an agentic assign-and-open-a-PR flow in preview, and OpenAI has opened gated access to cyber-tuned models through its Trusted Access for Cyber programme. Visa's open-source Vulnerability Agentic Harness brings this into the development pipeline: a chain of agents that discovers a vulnerability, verifies it, proposes a fix and validates the result, so code is hardened as it is written rather than audited after release.

We place this in Assess: the strongest tooling is still gated or in preview, and the patterns for folding it into a security programme are only forming. Regulated teams should watch closely and run the generally available pieces, such as Copilot Autofix where code scanning is already in place. Two cautions for regulated code: proprietary source sent to a hosted model leaves your boundary, so prefer self-hosted or on-premise options such as Visa's harness; and an agent-authored fix is a proposal, not a merge, so keep a person in the loop rather than auto-applying patches.

See also: AI red teaming tools, AI supply-chain security, AI-assisted code migration.

AI supply-chain security

AI systems have inherited every software-supply-chain risk and added a few of their own. The LiteLLM package compromise in March 2026 shows the inherited kind: a poisoned Trivy scanner, running unpinned in CI, stole the project's PyPI publish token and pushed backdoored releases with a credential-stealer inside. It was an ordinary pipeline attack that happened to land on AI infrastructure, a reminder that the AI stack is now a target in its own right. The newer risks sit around the models and the tooling. On the model side, weight files can execute code when you load them, and models are pulled from hubs with no record of where they came from. On the tooling side, a self-propagating npm worm has shown how a coding assistant's MCP configuration can be poisoned with a rogue server, whose tool descriptions smuggle in prompt-injection instructions to exfiltrate credentials and spread.

The defences are maturing, and the starting point is to treat a downloaded model as an untrusted binary and prefer serialisation formats that don't run code on load. On top of that, keep a machine-readable inventory of the models and AI services in use: ML-BOM is now a standardised CycloneDX capability, and vendors such as Endor Labs add model discovery and

malicious-model detection to conventional software composition analysis. Signing goes further than an inventory, and OpenSSF's Model Signing specification, built on Sigstore, covers a model's weights, tokeniser, config and datasets as one verifiable unit, so you can confirm the files are the ones the publisher signed and nothing was swapped in transit or on a registry. Google and Kaggle have demonstrated it, though adoption across model hubs and CI is still thin.

We place this in Assess. Only a small part of this is AI-specific, but that part isn't covered elsewhere: AI governance platforms handle compliance rather than artefact integrity, and red teaming targets model behaviour. Start with model provenance and an inventory, then extend the supply-chain controls you already run to cover the AI stack rather than standing up something separate.

See also: Agentic vulnerability discovery, MCP, AI governance platforms.

Hold

Not recommended for new projects; better alternatives exist.

OpenClaw

OpenClaw is an open-source runtime for persistent, always-on AI agents that execute multi-step tasks by controlling a computer: clicking, typing, navigating applications and browsing the web. It supports Claude, DeepSeek and OpenAI as backends, and has spawned a wave of imitators. NVIDIA's NemoClaw wraps OpenClaw in the NVIDIA Agent Toolkit with sandboxed execution against Nemotron 3 Super models. Moonshot's KimiClaw runs natively on kimi.com with a community skill marketplace and persistent cloud memory.

The security model hasn't been figured out across any of these variants. Persistent agent runtimes grant broad computer access to AI agents processing potentially untrusted instructions. The same prompt injection vulnerabilities that affect all visual computer use agents apply here, amplified by the breadth of access and the always-on character of the deployment. An agent with permission to control your browser, email and file system has an enormous blast radius if compromised.

We don't recommend OpenClaw or its variants for new projects until the security model matures. Teams already using one should enforce strict sandboxing, limit accessible applications and maintain human oversight for actions involving sensitive data.

See also: Visual computer use agents, Agentic tool use, Software engineering copilots, CaMeL.

Platforms

Infrastructure and managed services that host and run AI workloads: cloud AI services, vector databases, model serving platforms and MLOps infrastructure.

Adopt

60. Foundation models
61. Weights & Biases
62. Temporal
63. Data pipeline orchestration tools
64. Cloud model hosting platforms

Trial

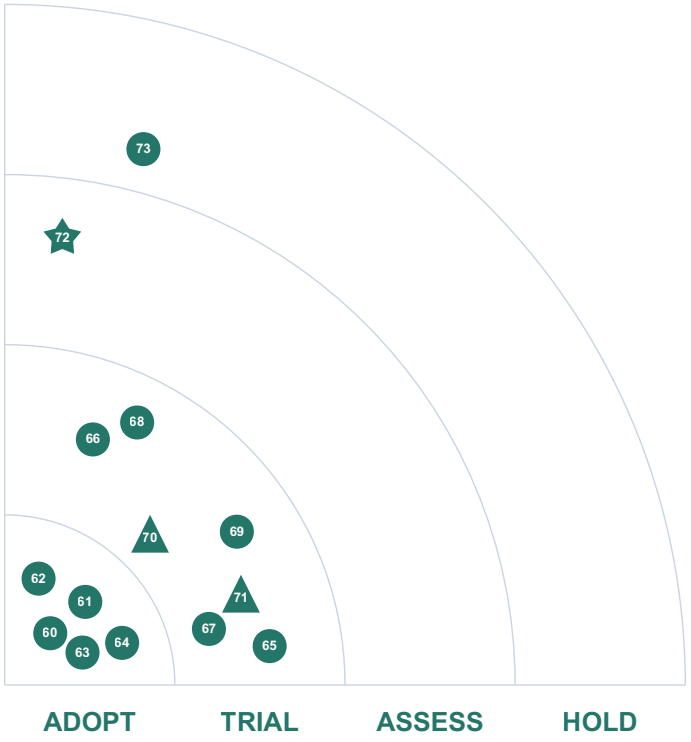
65. Production AI monitoring platforms
66. Open weight LLMs
67. Small language models
68. AI-powered workflow automation
69. Digital twin platforms
70. Agent memory architectures
71. AI governance platforms

Assess

72. Managed agent platforms

Hold

73. Building against vendor-specific APIs



Adopt

Mature, well-supported platforms ready for production use.

Foundation models

OpenAI, Anthropic, Google, Microsoft and Meta compete alongside a fast-growing open-weight field including DeepSeek, Alibaba, Moonshot AI and NVIDIA. Consolidation has begun too: Cohere and Aleph Alpha signed a definitive agreement in September 2026 to combine as a transatlantic sovereign-AI provider, subject to regulatory approval.

Providers differentiate across three tiers: smaller, faster models optimised for speed and cost; larger, more capable models balancing capabilities with response times; and specialised reasoning models for complex problem-solving. The distinction between general and reasoning models has largely dissolved, with the current flagships from OpenAI, Anthropic and Google integrating extended reasoning natively rather than exposing it through separate variants.

Foundation models warrant adoption for many business applications when paired with appropriate infrastructure (few-shot prompting, guardrails, RAG and evaluation frameworks). There is no universal "best model", so benchmark against your own use cases and weigh factors beyond raw performance: pricing, reliability, data privacy requirements and deployment options. High-quality open weight models provide additional options for organisations with specific security or deployment requirements, though licensing increasingly varies: several 2026 flagships ship under custom, revenue-gated terms rather than permissive open-source licences.

Key considerations

- **Performance & capabilities** (accuracy, speed, and domain-specific strengths)
- **Total cost of ownership** (API costs, compute resources, and integration)
- **Deployment options & technical requirements** (cloud, self-hosted, edge)
- **Data privacy & compliance** (regulatory, legal, and security implications)
- **Integration & lifecycle management** (context limitations, version control, updates)
- **Vendor stability & support** (roadmap alignment, documentation, community)

Foundation model providers feature comparison (October 2026)

Provider	Open Weights	Licence	Enterprise Focus	Edge Deployment	Long Context	Embedding API	Agentic Workflows	Model Selection Link
Ai2	✓	Permissive						Models
AI21 Labs	✓	Custom	✓	✓	✓		✓	Models
Alibaba	✓	Permissive		✓	✓	✓	✓	Models
Anthropic		Proprietary	✓		✓		✓	Models
AWS		Proprietary	✓		✓	✓	✓	Models
Baidu	✓	Permissive	✓	✓		✓	✓	Models
ByteDance	✓	Permissive	✓	✓	✓	✓	✓	Models
Cohere	✓	Permissive	✓		✓	✓	✓	Models
DeepSeek	✓	Permissive		✓	✓		✓	Models
Google	✓	Custom	✓	✓	✓	✓	✓	Models
IBM	✓	Permissive	✓	✓	✓	✓	✓	Models
Liquid AI	✓	Custom		✓			✓	Models
Meta	✓	Custom		✓	✓		✓	Models
Microsoft	✓	Permissive	✓	✓	✓		✓	Models
MiniMax	✓	Custom		✓	✓		✓	Models
Mistral AI	✓	Custom	✓	✓	✓	✓	✓	Models
Moonshot AI	✓	Custom	✓		✓		✓	Models
NVIDIA	✓	Custom	✓	✓	✓	✓	✓	Models
OpenAI	✓	Permissive	✓	✓	✓	✓	✓	Models
Reka AI	✓	Permissive	✓	✓			✓	Models
Stability AI	✓	Custom		✓				Models
Tencent	✓	Permissive			✓		✓	Models
X	✓	Custom	✓		✓		✓	Models
Zhipu AI	✓	Permissive	✓		✓		✓	Models

The capability gap between closed and open weights has narrowed to the point where it rarely decides the choice on its own. OpenAI, Anthropic and Google still set the frontier, and Microsoft now trains its own MAI models instead of relying on its OpenAI partnership alone. The open-weight field has drawn level on the coding and reasoning work our clients run day to day: DeepSeek, Alibaba's Qwen, Moonshot's Kimi and Tencent's Hunyuan are all competitive against

those benchmarks, and Meta has rejoined them with Muse Glimmer. NVIDIA is the one to watch: its Nemotron models are open, built for agentic work, and increasingly a base other labs fine-tune rather than a product chasing end users.

Open weights no longer guarantees permissive terms. Several of the strongest releases, Meta's Llama and Moonshot's Kimi K3, now ship under custom licences with revenue or user thresholds rather than Apache or MIT. Weigh the licence terms alongside the benchmark scores and your own testing before deciding whether a model fits your use case. For high stakes applications you may also want to check provenance: Ai2's OLMo publishes its training data and code alongside the weights, so you can see exactly what went into the model. At the small end, Liquid AI and the sub-10B models have made on-device inference workable where data cannot leave the device.

National and regional programmes are the last group, and most compete on language coverage and jurisdiction rather than capability. A growing number post-train a shared open base instead of pretraining from scratch, layering local language, data residency and governance on top. Singapore's SEA-LION builds on Gemma and Llama, and NVIDIA is positioning Nemotron as a base for sovereign builders. At the time of writing, two stand out for their unusual engineering: TII's Falcon-H1 runs attention and Mamba-2 state-space heads in parallel within each layer, Aleph Alpha's T-Free line drops the fixed token vocabulary altogether. Both are worth watching, especially for teams with data-residency requirements.

Feature definitions

- **Open Weights**: Models whose weights are publicly available for download and customisation
- **Licence**: How the open weights are licensed. Permissive means an OSI-approved licence (Apache 2.0 or MIT) with no revenue or user gate. Custom means open weights under bespoke terms, such as revenue thresholds, user caps or acceptable-use conditions. Proprietary means no public weights, available through an API only. Where a provider ships a mix, the classification follows its flagship open model.
- **Enterprise Focus**: Strong emphasis on governance, security, and enterprise integration
- **Edge Deployment**: Optimised for deployment on edge devices or resource-constrained environments
- **Long Context**: Support for context windows of 250K tokens or more
- **Embedding API**: Dedicated text embedding models and APIs for generating vector representations of text for semantic search and similarity tasks
- **Agentic Workflows**: Ability to autonomously plan and execute multi-step tasks using tools and external services. Goes beyond basic function calling to include complex workflow orchestration, error handling, dynamic planning based on intermediate results, and completing entire business processes without human intervention at each step

Weights & Biases

Weights & Biases tracks and visualises machine learning experiments, which helps when managing complex models and large datasets. By making experiment tracking light touch, needing just a few lines of code, it removes the friction that sometimes stops teams keeping up good measurement practices. CoreWeave acquired it in May 2025, and our recommendation stands, though we're watching how the product evolves under new ownership.

Shared dashboards and reports make results visible to the whole team, so experiments become shared assets rather than knowledge siloed in individual notebooks, which speeds up iteration. The benefits only arrive, though, where a team already values measurement and experimentation.

Temporal

Temporal is a workflow orchestration platform that provides durable execution for long-running processes. Although not AI-specific, it has become increasingly relevant as organisations build production agentic systems that must survive failures and run reliably over extended periods.

Durable execution means that if a multi-step process fails halfway through, it resumes from exactly where it left off. This differs from Kubernetes, which restarts crashed containers but knows nothing about application state, and it differs from checkpointing as used by LangGraph, CrewAI and Google's ADK. A saved checkpoint gives you no guarantee that the run finishes, that it recovers cleanly after a crash, or that it avoids repeating work already done. Temporal remembers that your workflow was on step 5 of 10, waiting for human approval, with specific context variables intact. The two are complementary, each handling failures at its respective layer.

Temporal's programming model treats workflows as ordinary code in familiar languages (Go, Java, Python, TypeScript, .NET) rather than configuration or visual diagrams. For agentic systems specifically, it addresses failure modes that many agent frameworks ignore: LLM calls timing out, tool invocations needing retry with backoff, human approvals taking days. Built-in retry policies and the ability to pause workflows indefinitely handle these cleanly.

Other durable execution frameworks are available, such as Restate, DBOS and Inngest, and its LangGraph plugin, in public preview since mid-2026, runs LangGraph nodes as Temporal activities so the agent stays ordinary code while durability lives underneath.

We recommend Temporal for teams moving beyond prototype agents towards production deployments that have to run reliably. For simpler use cases, lighter-weight alternatives may suffice.

Data pipeline orchestration tools

Data pipeline orchestration has become essential infrastructure for managing complex data workflows, particularly those supporting AI initiatives. Whilst transformation tools such as dbt handle the "what" of data processing, orchestration platforms manage the scheduling, execution and monitoring of entire pipelines.

Apache Airflow is the established standard, with broad integration support across cloud platforms, though operating it well takes deliberate investment in DAG conventions and tooling. Prefect emphasises developer experience and dynamic workflow adaptation, with faster development cycles but fewer third-party integrations. Dagster takes an asset-centric approach where data assets become first-class citizens, providing built-in lineage tracking and data quality monitoring.

The choice depends on organisational context. Established enterprises with diverse toolchains often gravitate towards Airflow's ecosystem breadth, teams prioritising developer velocity prefer Prefect, and organisations with complex lineage requirements consider Dagster's asset-aware approach.

Cloud model hosting platforms

Model hosting has evolved beyond simple API access, with distinct platforms serving different needs from prototyping to enterprise production. Cloud-based hosting has become the default for most AI deployments.

Enterprise production environments typically use AWS Bedrock, Google Vertex AI or Azure OpenAI Service, which provide fine-tuning capabilities with enterprise security and integration with existing cloud infrastructure. For performance-critical applications, specialised providers such as Fireworks AI and Together AI focus on inference optimisation and custom model deployment, though teams must weigh simplified deployment against reduced ecosystem integration.

A separate class of providers now competes on latency. Groq and Cerebras run inference on purpose-built silicon, serving a small catalogue of open-weight models at token throughput the general-purpose clouds don't match. The gain shows up in interactive and agentic workloads, where one task makes many sequential model calls and each round of latency adds up. The trade-off is coverage: fewer models, and little of the governance and integration tooling the larger platforms provide.

Development teams and startups often prefer Replicate, Modal or Hugging Face Inference Endpoints, which offer direct paths from trained model to production API with flexible pricing, at the cost of more limited enterprise governance.

Trial

Promising platforms with growing adoption, worth exploring for specific use cases.

Production AI monitoring platforms

Whilst experiment tracking tools such as Weights & Biases and MLflow excel at managing the development lifecycle, a distinct category has emerged to monitor AI systems in production. These tools detect drift and unexpected behaviour in deployed models that only surface when models encounter real-world data at scale.

Arize AI provides unified observability across traditional ML and LLM applications, continuously tracking feature and embedding drift (its open-source Phoenix handles run-level tracing; the platform adds drift monitoring at scale). Evidently AI offers both an open-source library and cloud platform, with over 100 metrics covering data quality and drift monitoring.

The payoff is early detection: you learn about performance degradation before it surfaces in support tickets. For teams already practising observability, AI-specific monitoring represents a natural extension of existing practices. This is the post-deployment counterpart to gating a release on evals: the gate catches regressions before they ship, monitoring catches the drift that only appears once live traffic arrives.

Open weight LLMs

Open weight models are now good enough for most production coding and reasoning, so teams usually choose one for control rather than capability. When you run the model yourself, your prompts and outputs stay inside the organisation, you decide when the version changes, and you can fine-tune it on your own data. For teams working with regulated or sensitive data, that control is usually the deciding factor.

You don't have to self-host to get this. Inference providers such as Fireworks AI and Baseten will run open-weight models on infrastructure you control, under a private contract and often with no data retention, and because they carry no frontier training costs the price per token is usually lower than the proprietary APIs. Running your own GPUs takes serious ML engineering and is rarely cheaper once that work is counted, so most teams are better off starting with a managed open-weight endpoint and self-hosting only when they need to control the full stack.

Small language models

Small language models (SLMs) are the smaller cousins of the open weight LLMs above. The current crop sits in the 1B–7B parameter range and runs comfortably on a single GPU, a modern laptop, or in some cases a phone. Microsoft's Phi-4-Mini (3.8B), Google's Gemma 4 E2B and E4B

(2–4B effective parameters), Meta's Llama 3.2 1B and 3B and Mistral's Ministral variants, joined by Liquid AI's LFM2.5 series engineered specifically for on-device inference, now perform at a level that was out of reach at this size two years ago.

In our experience, the case for SLMs is strongest for organisations with their own data to fine-tune on. A few thousand labelled examples and a fine-tuning run produce a model that has the organisation's terminology and edge cases built in. Hosted in-house, the data stays within the organisation, and the resulting model is small enough to deploy on the infrastructure the team already runs rather than dedicated hardware.

The natural fit is classification work where the language is nuanced and a wrong call is costly: routing customer messages by intent, triaging support tickets, tagging documents against a controlled vocabulary or extracting structured fields from free text. These are jobs that traditional keyword and feature-engineered approaches have never handled well, and where reaching for a frontier API on every request is overkill on both cost and latency. A fine-tuned SLM responds in the low hundreds of milliseconds or less and improves as more of the organisation's examples are added.

The trade-off is scope and discipline. SLMs will not match frontier models on open-ended reasoning or breadth of world knowledge, and getting them to behave reliably means taking the task definition, the labelling pipeline and evaluation seriously rather than leaning on clever prompting. For teams prepared to invest the time, the result is a model that is cheap to run, fits the infrastructure they already have and is trained on the data they already own.

See also: Open weight LLMs, Local model execution environments.

AI-powered workflow automation

Visual workflow automation platforms allow teams to build AI-powered business processes through drag-and-drop interfaces rather than code. Zapier focuses on connecting SaaS applications with AI capabilities. n8n offers self-hosting, open-source licensing and extensive customisation for technical teams. Microsoft Power Automate provides native Office 365 integration with enterprise governance. Make.com emphasises sophisticated visual workflow design with AI agent functionality.

Common use cases in banking include onboarding document checks, statement and form data extraction, support ticket triage and transaction categorisation.

When evaluating, consider technical capability, data sovereignty requirements and scalability. Self-hosted solutions such as n8n offer maximum control but require expertise, while SaaS offerings reduce overhead but may have cost implications at scale. Teams should also assess error recovery and debugging capabilities, as AI components can fail less predictably than traditional integrations.

Digital twin platforms

A digital twin is a working model of a system that stays in step with live data, so you can test a change against it and see the effect before rolling it out. The most developed versions are physical, like NVIDIA Omniverse, which simulates factories and logistics networks in enough detail to plan from. Few banks run that kind of infrastructure, so the version that reaches this audience models a business process or the organisation itself rather than a plant.

A twin of a loan or onboarding workflow lets you find where it stalls and try a redesign before it affects production, and the same approach run against a portfolio shows how it copes under a rate shock or a liquidity squeeze, the kind of stress a regulator will eventually apply. Platforms such as Simudyne build these as agent-based simulations, where each customer or counterparty is represented as an agent and the behaviour you care about emerges from how they interact. This works best on top of process mining, which reconstructs the current workflow from event logs, so what you simulate against reflects how the process runs in practice.

We keep this in Trial because the platform is rarely the thing that stops a project. A twin is only as useful as the live data feeding it and the model behind it, and in a regulated firm a simulation that informs a lending or capital decision falls under the same model-risk governance and audit requirements as anything else the firm relies on. Most of the effort goes into the data integration and the validation needed before anyone can act on the result, which is why few of these programmes have moved past pilots.

Agent memory architectures

Stateless agents hit a wall quickly. An AI coding assistant that forgets what it learned yesterday or a customer service agent that can't recall a conversation from last week force users to re-establish context on every interaction. Agent memory architectures address this with tiered storage: working context for the current session, long-term memory for persistent knowledge and episodic recall for past interactions. Over the past year memory has moved from a nice-to-have to a first-order design decision for anyone running agents beyond a single session.

Mem0 has gained the most traction, and ships as a native integration for AWS's Strands Agents SDK, one of several supported memory backends. It sits alongside your agent framework, automatically extracting and retrieving memories without changes to orchestration logic. Letta (formerly MemGPT) pioneered treating context management like virtual memory, paging information in and out as needed. Zep differentiates on temporal accuracy, maintaining a knowledge graph that tracks how facts change over time. A theme this year is the move away from pure vector recall towards schema- and graph-grounded memory, which retrieves by structure rather than similarity alone and tends to be easier to audit.

Persistent memory is also an attack surface, and a nastier one than a stray prompt. Adversarial content written once into persistent memory can outlast the session then surface weeks later. OWASP now tracks memory and context poisoning separately from prompt injection. The

defences differ from request-time filtering: scan content before it is written, make writes permission-aware, keep provenance on memory entries, and watch retrieval for behavioural drift. None is production-proven yet and reported attack success rates run high, so treat what an agent remembers with the same suspicion as any other untrusted input.

The frontier is memory that improves the agent. Systems such as Google's ReasoningBank turn an agent's own successes, and more usefully its failures, into reusable lessons retrieved on later tasks, so the agent gets better at the work it repeats. It is early, but it points at where memory is heading: not just recall, but steady improvement from experience.

For regulated industries, agent memory introduces governance questions your existing data policies may not cover. Where does the memory reside? How do you audit what an agent "knows" about a customer? How do you comply with deletion requests? Involve compliance and data protection teams early.

See also: Agentic tool use, Harness engineering, Microsoft Agent Framework, Agent identity, CaMeL, RAG.

AI governance platforms

The EU AI Act is phasing in. Its high-risk obligations, conformity assessments, risk documentation, human oversight and ongoing monitoring, were pushed to December 2027 by the Digital Omnibus, but the programmes to meet them take years to build. Banks and insurers already operate under model risk management frameworks (SR 11-7 in the US, SS1/23 in the UK), but these were written for statistical models with stable inputs and deterministic outputs. Extending them to LLM-based systems requires tooling that existing GRC platforms were not designed to provide.

Credo AI has emerged as the leading dedicated platform, providing AI risk classification aligned to the EU AI Act, NIST AI RMF and ISO 42001. It powers the compliance accelerators in IBM watsonx.governance through an OEM partnership, and Microsoft is integrating it with Azure AI Foundry. Others compose governance workflows from existing tools: model cards in Hugging Face, risk registers in GRC platforms, audit trails from observability tooling. Dedicated platforms offer structured workflows out of the box, while bespoke approaches require more engineering but can match how your organisation already works.

We move this to Trial. The space is young and consolidating fast, but the tooling is now worth the time to evaluate, particularly for financial services firms with EU AI Act obligations, even as the vendors keep shifting under them.

See also: LLM reproducibility, Production AI monitoring platforms.

Assess

Emerging or specialised platforms that merit evaluation for specific use cases.

Galileo

Cisco acquired Galileo in 2026 and folded it into Splunk Agent Observability, so the independent evaluation tool we assessed no longer stands alone. For current options see LLM observability tools; for open, self-hostable tooling, Langfuse or Phoenix.

Managed agent platforms

The counterpart to building your own harness is renting one. Managed agent platforms run the agent for you, offering a hosted harness with sandboxed code execution, persistent memory, scoped permissions and durable long-running sessions behind an API. AWS Bedrock AgentCore is the clearest example, a serverless, framework-agnostic runtime that hosts an agent built in any stack on isolated microVMs. Anthropic's Managed Agents and Google's Managed Agents take a more opinionated line, provisioning a sandbox where their own agent reasons, runs code and browses, while Vercel's Agent Stack assembles the same pieces from durable functions and sandboxes. Coding-specific services such as Cursor's cloud agents sit alongside, and Temporal provides the durable-execution layer several of these build on.

The trade is the familiar one: time-to-production against control and portability. A managed platform removes the undifferentiated work of running sandboxes, memory and retries, at the cost of tying your agent to a vendor's runtime and pricing, typically billed per token plus a session-hour fee.

A hosted agent that runs code and browses still inherits the unsolved problem of prompt injection and the trust boundary now runs through a third party's runtime. Least privilege still applies, and is harder to see from the outside: what credentials does the sandbox hold, and what can it reach on your behalf? For regulated teams there's a data-sovereignty question too, since prompts, retrieved context and tool traffic all cross into the vendor's infrastructure, which has a bearing on where regulated data is allowed to go.

We place managed agent platforms in Assess. The category is real and moving quickly, but most offerings reached preview or general availability only this quarter, pricing is unsettled, and portability between them is unproven. AgentCore is the most production-ready for teams that want to move sooner. Self-hosted, single-user runtimes such as OpenClaw sit at the opposite end and stay in Hold.

See also: Harness engineering, Temporal, Agent code sandboxes, AI-powered workflow automation, Vendor agent SDKs.

Hold

Not recommended for new projects; better alternatives exist.

Building against vendor-specific APIs

Tightly coupling applications to vendor-specific LLM APIs is risky in a market where capabilities and pricing shift monthly. Organisations that build directly against OpenAI, Anthropic or other proprietary APIs risk finding themselves locked in, facing painful migrations when a better or more cost-effective model emerges.

We recommend abstraction libraries that provide a common interface to multiple providers. AISuite or Simon Willison's LLM CLI let you switch between models with minimal code changes, handling the nuances of different APIs behind a consistent interface. These abstractions add some complexity and may limit access to vendor-specific features, but the protection against lock-in outweighs these drawbacks in most cases.



juxt.pro/ai-radar

The information in this document is provided as is and does not warrant the accuracy, completeness and fitness for a particular purpose. In no event shall Grid Dynamics Holdings, Inc be liable for any damages whatsoever arising from your reliance on any information contained in this document. Except as permitted under the Copyright Act no part of the document may be reproduced, transmitted, stored in a retrieval system, or translated into any human or computer language in any form by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise without the expressed permission of Grid Dynamics Holdings, Inc.